

Università degli Studi “Tor Vergata”

Corso di Laurea in Informatica

Corso di Basi di Dati e di Conoscenza

Esame del 24 febbraio 2025

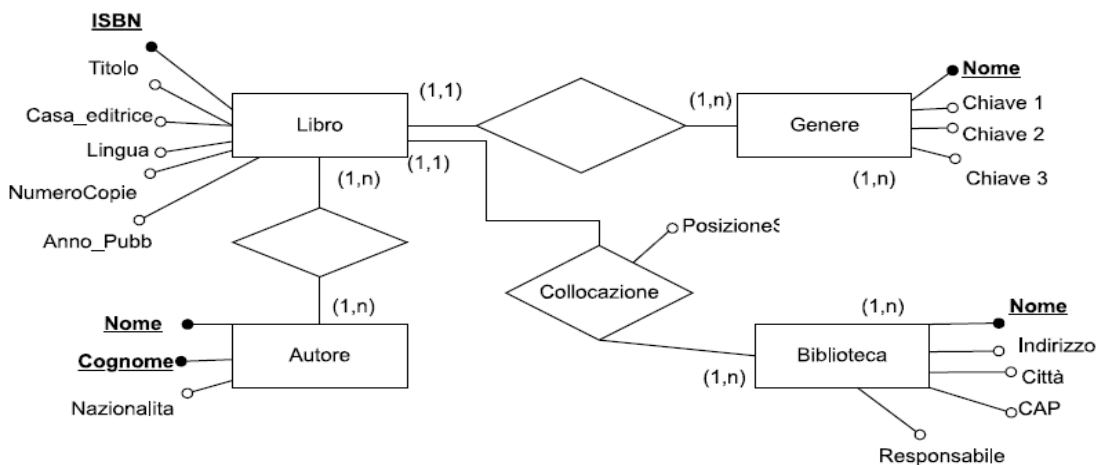
Domanda 1 (20% della valutazione complessiva)

Mostrare uno **schema concettuale ER** che rappresenti un database dei **Libri** presenti nelle diverse biblioteche di area dell’Ateneo di Tor Vergata

- Ogni **Libro** è identificato dal titolo, dall’ISBN (identificatore univoco), casa editrice, anno pubblicazione, lingua, numero copie.
- **Generi delle pubblicazioni**, nome del genere, fino a tre chiavi descrizione del genere.
- Ogni **Autore** è identificato dal nome, cognome, e nazionalità.
- Ogni **Biblioteca d’area** ha un nome (Biblioteca di Macroarea di Scienze, ingegneria, etc.), un indirizzo, comprensivo di città e CAP e il nome del responsabile;
- Ogni **Libro** è di un certo **genere** e di uno o più **autori** ed è collocato in una ed una sola biblioteca d’area;
- La relazione fra libro e Biblioteca d’area deve indicare **la collocazione** specifica

Mostrare lo **schema relazionale** derivante dallo schema concettuale.

Soluzione



LIBRO (ISBN, Titolo, CasaEditrice, AnnoPubblicazione, Lingua, NumeroCopie, NomeBiblioteca)
GENERE (NomeGenere, ChiaveDescr1, ChiaveDescr2, ChiaveDescr3)
LIBRO_GENERE (ISBN, NomeGenere)
AUTORE (IDAutore, Nome, Cognome, Nazionalità)
LIBRO_AUTORE (ISBN, IDAutore)
BIBLIOTECA (NomeBiblioteca, Indirizzo, Città, CAP, Responsabile)
COLLOCAZIONE (ISBN, NomeBiblioteca, PosizioneScaffale)

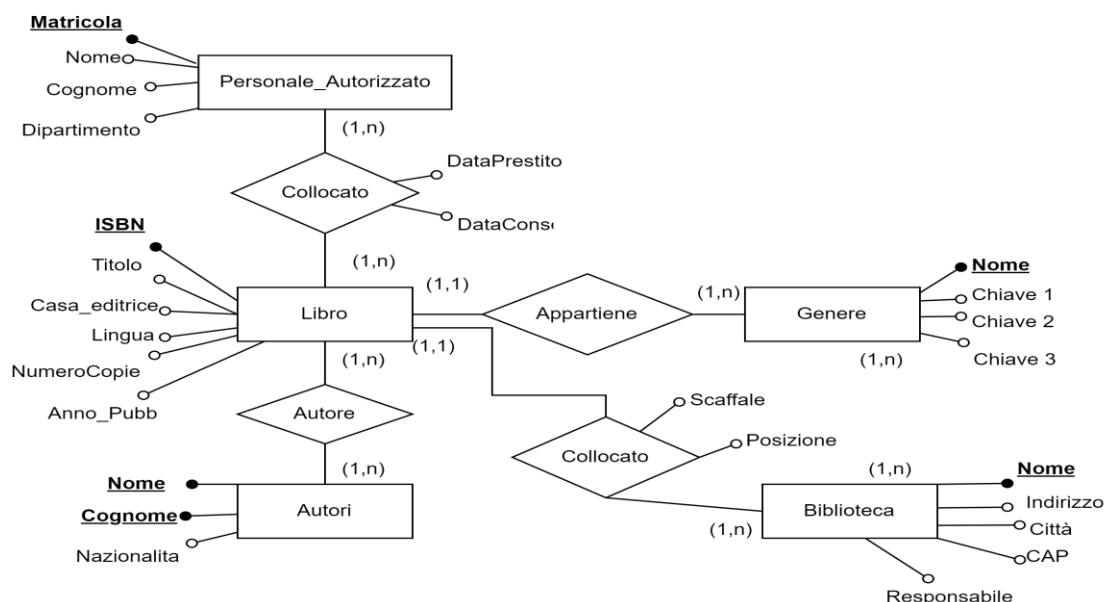
Domanda 2 (10% della valutazione complessiva)

Modificare lo schema in modo che rappresenti il **prestito dei libri** e vengano soddisfatte le seguenti specifiche:

- Ogni libro è dato in prestito ad un **personale autorizzato**, compatibilmente con il numero di copie disponibili.
- Ogni **personale autorizzato** è identificato dalla **matricola**, nome cognome e dipartimento di appartenenza.
- La relazione fra **libro** e **prestito** deve indicare la data di prestito e eventualmente la data di riconsegna.
- Ugni **personale autorizzato** non può prendere in prestito più di 5 libri.

Mostrare lo **schema relazionale** derivante dallo schema concettuale.

Soluzione



LIBRO (ISBN, Titolo, CasaEditrice, AnnoPubblicazione, Lingua, NumeroCopie, NomeBiblioteca)
GENERE (NomeGenere, ChiaveDescr1, ChiaveDescr2, ChiaveDescr3)
LIBRO_GENERE (ISBN, NomeGenere)
AUTORE (IDAutore, Nome, Cognome, Nazionalità)
LIBRO_AUTORE (ISBN, IDAutore)
BIBLIOTECA (NomeBiblioteca, Indirizzo, Città, CAP, Responsabile)
COLLOCAZIONE (ISBN, NomeBiblioteca, PosizioneScaffale)
PERSONALE_AUTORIZZATO (Matricola, Nome, Cognome, Dipartimento)
PRESTITO (ISBN, Matricola, DataPrestito, DataRiconsegna)

Vincoli di integrità:

- **NumeroCopie > 0** in LIBRO per permettere il prestito.
- **CHECK (COUNT(ISBN) <= 5)** per rispettare il limite massimo di 5 libri in prestito per persona.

Domanda 3 (30% della valutazione)

In base allo schema relazionale della domanda 2, scrivere le query in SQL che rispondano alle seguenti domande.

- Mostrare tutti i libri attualmente in prestito
- Elencare i libri per ogni genere e per ogni macroarea.
- Trovare il genere maggiormente dato in prestito.

Scrivere in algebra relazionale una query che mostri i libri di genere “fisica” nella biblioteca della Macroarea di Scienze.

Soluzioni

- Mostrare tutti i libri attualmente in prestito

```

SELECT L.ISBN, L.Titolo, L.CasaEditrice, L.AnnoPubblicazione,
L.Lingua, PA.Matricola, PA.Nome, PA.Cognome
FROM LIBRO L
JOIN PRESTITO P ON L.ISBN = P.ISBN
JOIN PERSONALE_AUTORIZZATO PA ON P.Matricola = PA.Matricola
WHERE P.DataRiconsegna IS NULL;
  
```

- Elencare i libri per ogni genere e per ogni macroarea.

```

SELECT G.NomeGenere, B.NomeBiblioteca AS Macroarea, L.ISBN, L.Titolo
FROM LIBRO L
JOIN LIBRO_GENERE LG ON L.ISBN = LG.ISBN
  
```

```

JOIN GENERE G ON LG.NomeGenere = G.NomeGenere
JOIN BIBLIOTECA B ON L.NomeBiblioteca = B.NomeBiblioteca
ORDER BY G.NomeGenere, B.NomeBiblioteca;

```

b. Trovare il genere maggiormente dato in prestito.

- Con subquery

```

SELECT G.NomeGenere, COUNT(*) AS NumeroPrestiti
FROM PRESTITO P
JOIN LIBRO L ON P.ISBN = L.ISBN
JOIN LIBRO_GENERE LG ON L.ISBN = LG.ISBN
JOIN GENERE G ON LG.NomeGenere = G.NomeGenere
GROUP BY G.NomeGenere
HAVING COUNT(*) = (
    -- Trova il massimo numero di prestiti per un singolo genere
    SELECT MAX(NumeroPrestiti)
    FROM (
        -- Conta i prestiti per ogni genere
        SELECT LG.NomeGenere, COUNT(*) AS NumeroPrestiti
        FROM PRESTITO P
        JOIN LIBRO L ON P.ISBN = L.ISBN
        JOIN LIBRO_GENERE LG ON L.ISBN = LG.ISBN
        GROUP BY LG.NomeGenere
    )
)
ORDER BY NumeroPrestiti DESC;

```

- Con Viste

Creiamo due viste intermedie per rendere più leggibile la query:

Passo 1: Creare una vista per contare i prestiti per ogni genere

```

CREATE VIEW Vista_Prestiti_Per_Genere AS
SELECT LG.NomeGenere, COUNT(*) AS NumeroPrestiti
FROM PRESTITO P
JOIN LIBRO L ON P.ISBN = L.ISBN
JOIN LIBRO_GENERE LG ON L.ISBN = LG.ISBN
GROUP BY LG.NomeGenere;

```

Passo 2: Creare una vista per trovare il numero massimo di prestiti

```

CREATE VIEW Vista_Max_Prestiti AS
SELECT MAX(NumeroPrestiti) AS MaxPrestiti
FROM Vista_Prestiti_Per_Genere;

```

Passo 3: Usare le viste per ottenere il genere più prestato

```

SELECT VPG.NomeGenere, VPG.NumeroPrestiti
FROM Vista_Prestiti_Per_Genere VPG

```

```
JOIN Vista_Max_Prestiti VMP ON VPG.NumeroPrestiti = VMP.MaxPrestiti
ORDER BY VPG.NumeroPrestiti DESC;
```

Scrivere in algebra relazionale una query che mostri i libri di genere “fisica” nella biblioteca della Macroarea di Scienze.

```
Π_{ISBN, Titolo, CasaEditrice, AnnoPubblicazione, Lingua,
NumeroCopie} (
    σ_{NomeGenere = 'Fisica' ∧ NomeBiblioteca = 'Biblioteca di
Macroarea di Scienze'} (
        (LIBRO ⋈ LIBRO_GENERE ⋈ GENERE) ⋈ BIBLIOTECA
    )
)
```

Domanda 4 (40% della valutazione)

Considerare il seguente schema relazionale NON in 1NF, 2NF e 3NF.

1. **Dipendenti** (Matricola, Nome, Cognome, DataNascita, Indirizzo, Città, CAP, Telefono, E-mail, CodUfficio, NomeUfficio, TelefonoUfficio, CodProgetto, NomeProgetto, DataInizioProgetto, DataFineProgetto, RuoloNelProgetto)
2. **Uffici** (CodUfficio, NomeUfficio, Piano, Edificio, IndirizzoEdificio, CittàEdificio, Telefono, CodDipendenteResponsabile, NomeResponsabile, CognomeResponsabile)
3. **Progetti** (CodProgetto, NomeProgetto, DataInizio, DataFine, Budget, CodDipendenteAssegnato, NomeDipendente, CognomeDipendente, RuoloNelProgetto)

Domande:

1. Identificare e spiegare quali problemi di normalizzazione presenta questo schema (violazioni di 1NF, 2NF e 3NF).
2. Proporre una decomposizione dello schema per portarlo in 3NF, mantenendo la perdita minima di informazioni e preservando la dipendenza funzionale.
3. Spiegare come la nuova struttura migliora l'integrità e la coerenza dei dati.

Soluzione

Domanda 1: Identificare i problemi di normalizzazione

L'attuale schema relazionale presenta diverse violazioni delle forme normali:

Violazione della 1NF (Prima Forma Normale)

- Esistono attributi multivalore e ripetuti, ad esempio:
 - In **Dipendenti**, un dipendente può essere assegnato a più progetti, ma il codice progetto (CodProgetto) è memorizzato nella stessa tabella, portando a ripetizioni.
 - In **Progetti**, un progetto può avere più dipendenti assegnati, con dati ripetuti per ogni dipendente.
- Esistono attributi non atomici, come "NomeResponsabile" e "CognomeResponsabile", che dovrebbero essere separati e normalizzati.

Violazione della 2NF (Seconda Forma Normale)

- In **Dipendenti**, la chiave candidata è (Matricola, CodProgetto), ma gli attributi **Nome, Cognome, DataNascita, Indirizzo, Città, CAP, Telefono, E-mail** dipendono solo da **Matricola** e non dall'intera chiave.
- In **Progetti**, la chiave è (CodProgetto, CodDipendenteAssegnato), ma **NomeProgetto, DataInizio, DataFine, Budget** dipendono solo da **CodProgetto**, non dalla chiave completa.
- Gli stessi problemi si presentano negli Uffici.

Violazione della 3NF (Terza Forma Normale)

- In **Dipendenti**, l'attributo "TelefonoUfficio" dipende da "CodUfficio" e non dalla chiave primaria della tabella.
- In **Progetti**, "RuoloNelProgetto" dipende da "CodDipendenteAssegnato", che non è una chiave primaria.

Domanda 2: Decomposizione dello schema in 3NF

Per correggere i problemi e ottenere uno schema in 3NF, possiamo riorganizzare le tabelle come segue:

1. Dipendenti (Informazioni personali)

Dipendenti (Matricola, Nome, Cognome, DataNascita, Indirizzo, Città, CAP, Telefono, E-mail, CodUfficio)

- Qui abbiamo eliminato gli attributi relativi ai progetti e agli uffici che non appartengono direttamente alla relazione tra dipendenti.

2. Uffici (Informazioni sugli uffici)

Uffici (CodUfficio, NomeUfficio, Piano, Edificio (FK), Telefono, CodDipendenteResponsabile)

- La dipendenza del responsabile dell'ufficio è mantenuta tramite la chiave esterna "CodDipendenteResponsabile".

Edificio (Edificio, IndirizzoEdificio, CittàEdificio,)

3. Progetti (Informazioni sui progetti)

Progetti (CodProgetto, NomeProgetto, DataInizio, DataFine, Budget)

- Abbiamo rimosso gli attributi ridondanti relativi ai dipendenti.

4. PartecipazioneProgetti (Assegnazioni ai progetti)

PartecipazioneProgetti (Matricola, CodProgetto, RuoloNelProgetto)

- Questa tabella gestisce la relazione multi-a-molti tra dipendenti e progetti, con il ruolo specificato.

Passo 3: Benefici della normalizzazione

Dopo la normalizzazione, abbiamo:

1. **Eliminato ridondanze** → Nessuna informazione viene ripetuta in più tabelle.
2. **Aumentato l'integrità dei dati** → Le dipendenze funzionali sono rispettate, e i dati sono più facili da aggiornare.
3. **Migliorato l'efficienza dello storage** → Minore duplicazione dei dati riduce l'uso di spazio.
4. **Facilitato l'aggiornamento e la manutenzione** → Evitiamo problemi di inconsistenza quando vengono modificati i dati.

Conclusione

Ora lo schema è in 3NF e consente di gestire i dipendenti, gli uffici e i progetti in modo più strutturato, evitando anomalie di aggiornamento e riducendo la ridondanza.